Algorithm 3 Ullmann's Subgraph Matching Algorithm

```
1:  Input: query graph G_q = (V_q, E_q), data graph G = (V, E)
2:  generate matching order π = [u_1, u_2, ..., u_{|V_q|}]   (u_i ∈ V_q)
3:  ENUMERATE(∅, 1, π)
4:  procedure ENUMERATE(S, i, π)
5:      C_S(u_i) ← viable vertex candidates in G to match u_i
6:      for all v ∈ C_S(u_i) do
7:          append S with (u_i, v)
8:          if |S| = k then output S
9:          else ENUMERATE(S, i + 1, π)
10:         pop (u_i, v) from S
```

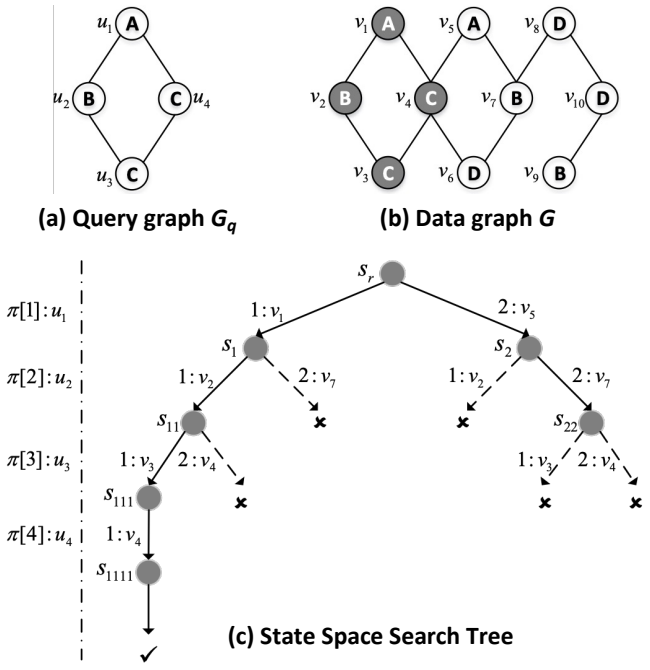

Figure 9: Illustration of Subgraph Matching

A Subgraph Matching

Subgraph matching (SM) is a representative SF problem that enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q. Many SM algorithms [36, 55] are based on Ullmann's backtracking framework [40], which orders query vertices and recursively extends a partial mapping while pruning infeasible candidates.

For completeness, we outline a basic Ullmann-style backtracking procedure in Algorithm 3. The algorithm incrementally matches data vertices to query vertices $u_1, \ldots, u_{|V_q|}$ according to a predefined matching order, maintaining the current partial mapping in S. At each recursion level, viable candidate vertices in G are derived for the next query vertex, and infeasible extensions are pruned early based on adjacency constraints. The search proceeds recursively until either a complete match is found or no further extension is possible.

This process explores a state-space tree as illustrated in Figure 9. A node at level i corresponds to a partial mapping that matches the query vertex prefix $[u_1, \ldots, u_i]$. For example, the partial mapping $S = [(u_1, v_5), (u_2, v_7), (u_3, v_4)]$ is pruned due to the absence of an edge (v_7, v_4) corresponding to the query edge (u_2, u_3). Various optimizations adopted by modern SM algorithms [36]—such as improving the matching order (Line 2) and tightening candidate sets (Line 5)—aim to reduce the search space but preserve this fundamental backtracking structure.

Importantly, the state-space tree explored by Ullmann-style algorithms is also a subgraph enumeration tree over G: each node represents a subgraph in G that matches a prefix of the query graph. Since the programming model of G-thinkerCG is built upon the subgraph enumeration tree and subgraph-task abstractions, it can naturally support CPU-GPU co-processing for SM algorithms by adapting their serial backtracking counterparts to the G-thinkerCG interface.

Table 3: Effect of System Parameters

Parameter	Value	soc-sinaweibo	wikipedia_link_sr
η / #{warps}	100	203.8	OOT
	500	113.9	542.3
	1000	**109.6**	**465.2**
	2000	231.0	948.0
CPU Chunk	1	117.1	357.6
	10	108.9	**316.1**
	50	**91.6**	465.2
	100	110.8	564.9
GPU Chunk	100K	143.3	511.9
	500K	**112.1**	**315.9**
	1M	159.4	1085.5
τ_c	1	220.3	567.3
	n_c / 2	**111.4**	**471.9**
	n_c / 4	112.5	476.7

B The Algorithm of `atomicSubNonNegative(.)`

To pop a subgraph from the stack $\mathcal{H}$, each thread of a warp calls the `pop_offsets(sglen)` operation shown in Figure 4-❺, which properly decrements `otail` in Line 2, and returns the trackers `[offsets[ot-2], ohead[ot-1])` for all threads of the warp to read the obtained subgraph from $\mathcal{H}$.`vertices`. Note that since we cannot decrement `otail` to be smaller than 0, we cannot use `atomicSub(.)` since when multiple warps subtract 2 from `otail` consecutively, `otail` can become negative. We solve this issue by calling `atomicSubNonNegative(.)` as shown in Figure 4-❻.

Specifically, Line 1 reads the value of `otail` into variable `old`, which is then read into `assumed` in Line 3. If this value is already 0 (Line 4), `otail` cannot be further decremented so Line 5 returns 0 directly as the original value of `otail`. Otherwise, Line 6 atomically checks if `otail`'s previously retrieved value (i.e., `old`) still equals `assumed`: (1) if so, no other warp has changed `otail` since Line 3, so the decremented value (`assumed−dec`) is written to `otail`; (2) while otherwise, `otail` has been updated by another warp, so we cannot update `otail` due to the atomicity requirement of subtraction, and `old` gets the latest value of `otail` in Line 6 and tries again until an atomic subtraction is successfully done (Line 7), after which the old value of `otail` before subtraction is returned in Line 8.

C System Parameter Study

We next present the effect of various system parameters on the running time of Hybrid using two representative datasets *soc-sinaweibo*

and *wikipedia_link_sr*. As shown in Table 3 (where OOT means running out of the maximum allowed time threshold of 1800 seconds), our default parameters generally achieve the best performance. Note that a tradeoff exists for each parameter: when η and vertex chunk sizes are too large, the task granularity is too coarse which hampers load balancing; while when η and GPU chunk size are too small, the computing power of GPU threads cannot be saturated, and when CPU chunk size is too small, the cost for CPU workers to wait in $\mathbb{W}$ increases. As for τ_c, if it is too small, GPU-to-CPU task stealing would be too aggressive and could lead to task thrashing if some CPU worker encounters a task timeout and adds many new tasks back to $\mathcal{S}$; while if it is too large, surplus GPU tasks cannot be timely rescheduled to CPU workers for processing. Our default parameters achieve a good tradeoff.